Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

MedLexer: Analizador Léxico para Gestión Hospitalaria

PONDERACIÓN: 25 pts

Tiempo estimado: 20 hrs

1. Marco Formativo

1.1. Valor

Nombre del Valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad Social	Los estudiantes demostrarán responsabilidad social al diseñar una herramienta que simula el procesamiento de datos médicos reales. Comprender que un error léxico en un sistema hospitalario puede tener consecuencias críticas fortalece la cultura de calidad y precisión en el desarrollo de software.

1.2. Competencia(s)

Tipo de Competencia	Descripción
Competencia General	Diseñar, implementar y evaluar soluciones computacionales aplicando los fundamentos teóricos y prácticos de las ciencias de la computación para resolver problemas reales en el ámbito del desarrollo de software.
Competencia Específica	Aplicar los conceptos de teoría de autómatas, lenguajes formales y expresiones regulares para desarrollar un analizador léxico en C++ que procese e interprete un lenguaje de dominio específico orientado a la gestión hospitalaria, implementando el AFD manualmente sin herramientas automáticas de generación léxica.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Desarrollar una aplicación de escritorio en C++ con interfaz gráfica (Qt o wxWidgets) que implemente un analizador léxico capaz de interpretar archivos. med con información hospitalaria, generando cuatro reportes HTML con estadísticas médicas y visualizaciones con Graphviz.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El analizador debe reconocer correctamente al menos el 95% de los tokens del lenguaje MedLang, detectar errores con posición exacta (línea y columna), generar los 4 reportes HTML y procesar archivos de hasta 1000 líneas en menos de 3 segundos.
Alcanzable (¿Cómo?)	Posible con los recursos disponibles.	Utilizando C++17 con STL, implementando el AFD manualmente con una función de transición de estados explícita, y usando Qt 5/6 o wxWidgets para la GUI. Los reportes se generan como archivos HTML con CSS embebido.
Realista (¿Para qué?)	Contribuye a metas más amplias.	Fortalecer competencias en teoría de autómatas, diseño de compiladores, programación orientada a objetos en C++ y desarrollo de GUIs, preparando al estudiante para los cursos de Compiladores 1 y 2.
A Tiempo (¿Cuándo?)	Tiene fecha límite definida.	Completar el proyecto en 4 semanas: Semana 1 diseño del lenguaje y AFD, Semana 2 implementación del motor léxico, Semana 3 GUI y reportes, Semana 4 pruebas y documentación.

2. Resumen Ejecutivo

MedLexer es un proyecto de desarrollo de software cuyo objetivo es construir una herramienta de escritorio en C++ que permita analizar, interpretar y reportar estructuras de datos médicos hospitalarios a partir de archivos de texto con extensión .med, estructurados bajo las reglas del mini-lenguaje MedLang. El proyecto se enfoca en la implementación manual de un Autómata Finito Determinista (AFD) para el análisis léxico, sin recurrir a herramientas automáticas de generación léxica ni a expresiones regulares de C++ para la tokenización.

La solución integra cuatro componentes principales: el motor léxico (AFD implementado en C++17 con STL), la interfaz gráfica de escritorio (Qt 5/6 o wxWidgets), el generador de cuatro reportes HTML con estadísticas médicas, y el módulo de visualización con Graphviz. El proyecto exige que el estudiante comprenda en profundidad la teoría de autómatas finitos y sea capaz de traducirla a código C++ eficiente, manejando correctamente todos los tipos de tokens del lenguaje MedLang incluyendo literales de fecha, literales de hora, códigos de identificación con guión y enumeraciones médicas.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

Los sistemas de gestión hospitalaria manejan grandes volúmenes de datos estructurados: registros de pacientes, perfiles de médicos especializados, agendas de citas y prescripciones médicas. Un error en el procesamiento léxico de estos datos puede tener consecuencias directas en la atención al paciente.

El problema que este proyecto busca resolver es la necesidad de una herramienta interactiva que permita comprender cómo se procesa léxicamente un lenguaje formal de dominio médico. La herramienta debe interpretar archivos .med escritos en el mini-lenguaje MedLang, detectar y reportar con precisión cualquier error léxico, y generar reportes visuales que exploten la información extraída para facilitar la gestión hospitalaria simulada.

3.2 Alcance del proyecto

Alcance obligatorio:

- Lenguaje MedLang con al menos 10 tipos de tokens: palabras reservadas (HOSPITAL, PACIENTES, MEDICOS, CITAS, DIAGNOSTICOS), códigos con guión (PAC-001, MED-023), literales de fecha (AAAA-MM-DD), literales de hora (HH:MM), enteros, cadenas de texto, enumeraciones de especialidad y dosis, tipos de sangre restringidos y delimitadores estructurales.
- AFD implementado manualmente en C++17 con la clase LexicalAnalyzer y el método nextToken(). Prohibido el uso de std::regex para la tokenización principal.
- Tabla de tokens: número, lexema, tipo de token, línea y columna con precisión del 100%.
- Tabla de errores léxicos: número, lexema inválido, tipo de error, descripción y posición exacta. El análisis debe continuar tras cada error sin detenerse.
- Cuatro reportes HTML con CSS embebido (descritos en detalle en la sección 3.4).
- Diagrama Graphviz (código DOT) con la jerarquía del hospital y relaciones paciente-médico-diagnóstico.
- Interfaz gráfica en Qt 5/6 o wxWidgets 3.x con todos los paneles requeridos.

Alcance opcional:

- Resaltado de sintaxis en tiempo real en la GUI.
- Exportación de la tabla de tokens a CSV o JSON.
- Detección automática de conflictos de horario entre citas del mismo médico.
- Estadísticas del análisis léxico: frecuencia de tokens por tipo, tiempo de análisis en ms.
- Soporte para analizar múltiples archivos .med en una misma sesión.

3.3 Estructura del Lenguaje MedLang

El archivo de entrada con extensión .med sigue una estructura jerárquica donde el elemento raíz HOSPITAL contiene cuatro secciones: PACIENTES, MEDICOS, CITAS y DIAGNOSTICOS.

Token / Elemento	Tipo	Descripción / Valores	Ejemplo
HOSPITAL	Palabra reservada raíz	Bloque principal del archivo. Sin él el archivo es inválido.	HOSPITAL { ... }
PACIENTES	Palabra reservada de sección	Inicia el bloque de registro de pacientes.	PACIENTES { paciente: ... }
paciente	Palabra reservada de elemento	Define un paciente con sus atributos.	paciente: "Ana López" [edad: 34, tipo_sangre: "O+"]
MEDICOS	Palabra reservada de sección	Inicia el bloque de registro de médicos.	MEDICOS { medico: ... }
medico	Palabra reservada de elemento	Define un médico con especialidad y código único.	medico: "Dr. Ruiz" [especialidad: CARDIOLOGIA, codigo: "MED-001"]
CITAS	Palabra reservada de sección	Inicia el bloque de citas médicas.	CITAS { cita: ... }
cita	Palabra reservada de elemento	Asocia paciente y médico con fecha y hora.	cita: "Ana López" con "Dr. Ruiz" [fecha: 2025-04-10, hora: 09:00]
DIAGNOSTICOS	Palabra reservada de sección	Inicia el bloque de diagnósticos y tratamientos.	DIAGNOSTICOS { diagnostico: ... }
diagnostico	Palabra reservada de elemento	Asocia condición, medicamento y dosis a un paciente.	diagnostico: "Ana López" [condicion: "Hipertensión", medicamento: "Losartán 50mg", dosis: DIARIA]
Especialidades	Enumeración	CARDIOLOGIA | NEUROLOGIA | PEDIATRIA | CIRUGIA | MEDICINA_GENERAL | ONCOLOGIA	especialidad: NEUROLOGIA
Frecuencia de dosis	Enumeración	DIARIA | CADA_8_HORAS | CADA_12_HORAS | SEMANAL	dosis: CADA_12_HORAS
Tipo de sangre	Cadena restringida	"A+" | "A-" | "B+" | "B-" | "O+" | "O-" | "AB+" | "AB-"	tipo_sangre: "AB+"
Fecha	Literal de fecha	Formato AAAA-MM-DD (mes 01-12, día 01-31).	fecha: 2025-03-15
Hora	Literal de hora	Formato HH:MM en 24 horas (00:00 - 23:59).	hora: 14:30
Entero	Literal numérico	Números enteros positivos. Usado en edad y habitación.	edad: 45, habitacion: 302
Cadena	Literal de texto	Texto entre comillas dobles.	condicion: "Artritis Reumatoide"
Código ID	Alfanumérico con guión	Código único del médico/paciente. Formato: 3 letras + guión + dígitos.	codigo: "MED-001"

Ejemplo de archivo de entrada válido:

```
HOSPITAL {
    PACIENTES {
        paciente: "María García"  [edad: 45, tipo_sangre: "A+", habitacion:
302],
        paciente: "Carlos Mendoza" [edad: 62, tipo_sangre: "O-", habitacion:
410],
    };
    MEDICOS {
        medico: "Dr. Roberto Juárez" [especialidad: CARDIOLOGIA, codigo:
"MED-001"],
        medico: "Dra. Ana Solís"     [especialidad: NEUROLOGIA,  codigo:
"MED-002"],
    };
    CITAS {
        cita: "María García" con "Dr. Roberto Juárez" [fecha: 2025-04-10,
hora: 09:00],
        cita: "Carlos Mendoza" con "Dra. Ana Solís"    [fecha: 2025-04-10,
hora: 11:30],
    };
    DIAGNOSTICOS {
        diagnostico: "María García" [condicion: "Hipertensión Arterial",
            medicamento: "Losartán 50mg", dosis: DIARIA],
    };
};
```

3.4 Reportes a Generar

El sistema debe generar automáticamente cuatro reportes HTML al finalizar el análisis léxico exitoso, aprovechando la información extraída de los tokens. Cada reporte se guarda como un archivo HTML independiente con CSS embebido, y debe poder abrirse directamente en cualquier navegador web. A continuación se describe el objetivo, el contenido y el formato esperado de cada reporte, acompañado de una tabla de ejemplo que ilustra cómo deben presentarse los datos.

Reporte 1 — Historial de Pacientes

Objetivo: Presentar el registro completo de todos los pacientes del hospital mostrando sus datos personales junto con el diagnóstico activo y el medicamento prescrito extraídos del bloque DIAGNOSTICOS. Si un paciente no tiene diagnóstico registrado, debe indicarse explícitamente. Cada fila debe incluir un indicador visual del estado del paciente basado en reglas que el estudiante defina en su lógica de generación de reportes (por ejemplo: ACTIVO si tiene diagnóstico, SIN DIAG. si no lo tiene, CRÍTICO si el médico lo ha marcado como urgente o tiene múltiples diagnósticos activos).

Contenido mínimo requerido por fila:

- Nombre del paciente, edad y tipo de sangre (del bloque PACIENTES).
- Condición médica activa y medicamento con frecuencia de dosis (del bloque DIAGNOSTICOS). Si no hay diagnóstico, mostrar un indicador claro.
- Estado visual del paciente con color diferenciado: ACTIVO (verde), SIN DIAG. (naranja), CRÍTICO (rojo).

Ejemplo del formato esperado:

Paciente	Edad	Sangre	Diagnóstico activo	Medicamento / Dosis	Estado
María García	45	A+	Hipertensión Arterial	Losartán 50mg / DIARIA	ACTIVO
Carlos Mendoza	62	O-	Diabetes Tipo 2	Metformina 850mg / CADA_12_HORAS	ACTIVO
Sofía Ramírez	29	B+	Sin diagnóstico registrado	—	SIN DIAG.
Pedro Castillo	71	AB-	Insuficiencia Cardiaca	Furosemida 40mg / DIARIA	CRÍTICO

Reporte 2 — Carga de Médicos por Especialidad

Objetivo: Mostrar el perfil de cada médico registrado indicando su especialidad, el número de citas programadas, el número de pacientes distintos atendidos, y un indicador visual del nivel de carga de trabajo. Este reporte permite identificar en forma inmediata qué especialidades están saturadas y cuáles tienen capacidad disponible. Los umbrales de carga son sugeridos (BAJA: 1-3, NORMAL: 4-6, ALTA: 7-8, SATURADA: 9+) y el estudiante puede ajustarlos con justificación en el manual técnico.

Contenido mínimo requerido por fila:

- Nombre del médico y código único (del bloque MEDICOS).
- Especialidad médica (de la enumeración del lenguaje).
- Total de citas programadas para ese médico (conteo cruzado con el bloque CITAS).
- Número de pacientes distintos que atiende (conteo de pacientes únicos en sus citas).
- Nivel de carga con color diferenciado: BAJA (azul), NORMAL (verde), ALTA (naranja), SATURADA (rojo).

Ejemplo del formato esperado:

Médico	Código	Especialidad	Citas Prog.	Pacientes	Nivel de Carga
Dr. Roberto Juárez	MED-001	CARDIOLOGIA	4	3	NORMAL
Dra. Ana Solís	MED-002	NEUROLOGIA	7	5	ALTA
Dr. Luis Herrera	MED-003	PEDIATRIA	2	2	BAJA
Dra. Carmen Vega	MED-004	CIRUGIA	9	7	SATURADA

Reporte 3 — Agenda de Citas con Detección de Conflictos

Objetivo: Presentar la agenda completa de citas ordenadas cronológicamente por fecha y hora. El sistema debe detectar y marcar visualmente los conflictos de horario: casos en que el mismo médico tiene dos citas programadas en la misma fecha y a la misma hora. Las filas con conflicto deben resaltarse con fondo rojo para identificarlas de inmediato. Esta es la funcionalidad de mayor valor práctico del sistema, ya que representa un error real que ocurre en hospitales.

Contenido mínimo requerido por fila:

- Fecha y hora de la cita (literales de fecha y hora del bloque CITAS).
- Nombre del paciente y nombre del médico asignado.
- Especialidad del médico (información cruzada con el bloque MEDICOS).
- Estado de la cita: CONFIRMADA (sin conflicto, fondo verde), ⚠ CONFLICTO (mismo médico, misma fecha y hora, fondo rojo) o PENDIENTE (fecha futura aún no confirmada, fondo naranja).

Ejemplo del formato esperado (la fila en rojo indica conflicto detectado):

Fecha	Hora	Paciente	Médico	Especialidad	Estado
2025-04-10	09:00	María García	Dr. Roberto Juárez	CARDIOLOGIA	CONFIRMADA
2025-04-10	09:00	Pedro Castillo	Dr. Roberto Juárez	CARDIOLOGIA	⚠ CONFLICTO
2025-04-10	11:30	Carlos Mendoza	Dra. Ana Solís	NEUROLOGIA	CONFIRMADA
2025-04-11	08:00	Sofía Ramírez	Dr. Luis Herrera	PEDIATRIA	PENDIENTE

⚠ Nota sobre la fila resaltada

En el ejemplo anterior, la segunda fila (Pedro Castillo — Dr. Roberto Juárez — 09:00 del 2025-04-10) está marcada como CONFLICTO porque el mismo médico ya tenía una cita asignada a las 09:00 de ese mismo día con María García. El sistema debe detectar esta colisión automáticamente al procesar el bloque CITAS, comparando médico, fecha y hora para cada par de citas.

Reporte 4 — Estadístico General del Hospital

Objetivo: Proveer un resumen ejecutivo con las estadísticas globales del hospital extraídas del archivo analizado. Este reporte se divide en dos secciones: un panel de indicadores clave (KPIs) con los datos más relevantes del archivo procesado, y una tabla de distribución de carga por especialidad con una barra de progreso visual que permite comparar visualmente el nivel de ocupación de cada servicio médico.

Sección A — Indicadores clave del hospital:

- Nombre del hospital (del token del bloque HOSPITAL, si está definido como atributo, o como nombre del archivo).
- Total de pacientes, médicos y citas extraídos de los respectivos bloques.
- Número de citas con conflicto de horario detectados.
- Porcentaje de pacientes con diagnóstico activo sobre el total registrado.
- Medicamento más frecuentemente prescrito y número de pacientes que lo reciben.
- Especialidad médica con mayor carga de citas y nombre del médico correspondiente.
- Promedio de edad de todos los pacientes registrados.

Ejemplo de la Sección A:

Indicador	Valor
Nombre del hospital	Hospital General San Carlos
Total de pacientes registrados	12
Total de médicos activos	6
Total de citas programadas	23
Citas con conflicto de horario	**2** ⚠
Pacientes con diagnóstico activo	9 de 12 (75%)
Medicamento más prescrito	Losartán 50mg (3 pacientes)
Especialidad con mayor carga de citas	CIRUGIA — Dra. Carmen Vega (9 citas)
Promedio de edad de los pacientes	51.8 años

Sección B — Distribución de carga por especialidad:

- Una fila por cada especialidad médica presente en el bloque MEDICOS.
- Número de médicos, citas totales y pacientes distintos por especialidad.
- Barra de carga visual (puede ser texto con caracteres ▉ o una barra HTML) que muestre el porcentaje de citas de esa especialidad sobre el total del hospital.
- Las especialidades con ocupación mayor al 80% deben resaltarse en rojo para alertar sobre saturación.

Ejemplo de la Sección B:

Especialidad	Médicos	Citas	Pacientes	Barra de ocupación
CARDIOLOGIA	1	4	3	40%
NEUROLOGIA	1	7	4	70%
PEDIATRIA	1	2	2	20%
CIRUGIA	1	9	5	90%
MEDICINA_GENERAL	2	3	3	30%

3.5 Tabla de Errores Léxicos

Cuando el archivo .med contiene tokens inválidos, el analizador debe acumular todos los errores encontrados durante el recorrido completo del archivo (modo de recuperación de errores) y mostrarlos al finalizar, tanto en la GUI como en un archivo HTML adicional de errores. El sistema no debe detenerse ante el primer error; debe continuar el análisis para reportar todos los problemas en una sola pasada.

Cada error en la tabla debe incluir:

- Número del error: identificador incremental.
- Lexema: el valor exacto que causó el error tal como apareció en el archivo fuente.
- Tipo de Error: token inválido, cadena sin cerrar, fecha con mes/día inválido, hora fuera de rango, especialidad no reconocida, tipo de sangre inválido o carácter ilegal.
- Descripción: explicación clara del error con indicación del valor correcto esperado.
- Línea y Columna: posición exacta del carácter que causó el error.
- Gravedad: ERROR (el análisis puede continuar) o CRÍTICO (el error impide procesar el bloque actual, como una cadena sin cerrar).

Ejemplo del formato de tabla de errores:

No.	Lexema	Tipo de Error	Descripción	Línea	Columna	Gravedad
1	"O++"	Tipo de sangre inválido	El valor 'O++' no es un tipo de sangre reconocido. Valores válidos: O+, O-, A+, A-, B+, B-, AB+, AB-.	8	22	**ERROR**
2	2025-13-01	Fecha con mes inválido	El mes '13' está fuera del rango permitido (01-12). Formato esperado: AAAA-MM-DD.	15	12	**ERROR**
3	"Cardio"	Especialidad no reconocida	'Cardio' no es una especialidad válida. Use: CARDIOLOGIA, NEUROLOGIA, PEDIATRIA, CIRUGIA, MEDICINA_GENERAL u ONCOLOGIA.	23	28	**ERROR**
4	"Losartán 50	Cadena sin cerrar	Se encontró el inicio de una cadena (comilla doble) pero no se detectó el cierre antes del fin de la línea.	31	41	**CRÍTICO**
5	25:00	Hora fuera de rango	La hora '25:00' está fuera del rango permitido (00:00 - 23:59). Formato esperado: HH:MM en 24 horas.	38	17	**ERROR**

3.6 Diagrama Graphviz

Adicionalmente a los cuatro reportes HTML, el sistema debe generar un archivo hospital.dot con código Graphviz que represente la estructura jerárquica y las relaciones del hospital analizado. El estudiante es libre de mejorar el diseño del diagrama con colores, formas de nodos y agrupaciones adicionales.

El diagrama debe incluir como mínimo:

- Un nodo raíz con el nombre del hospital.
- Cuatro nodos intermedios para las secciones (PACIENTES, MEDICOS, CITAS, DIAGNOSTICOS).
- Un nodo hoja por cada paciente y médico registrado con sus datos principales.
- Aristas desde CITAS que conecten el nodo del paciente con el nodo del médico de cada cita, etiquetadas con la fecha y hora.
- Aristas desde DIAGNOSTICOS que conecten el diagnóstico con el nodo del paciente correspondiente.

Código DOT de ejemplo para el archivo de entrada mostrado en la sección 3.3:

```
digraph Hospital {
    rankdir=TB; node [shape=box, style=filled, fontname="Arial"];
    H  [label="Hospital General San Carlos", fillcolor="#1A4731",
fontcolor=white, shape=ellipse];
    P  [label="PACIENTES",     fillcolor="#2E7D52", fontcolor=white];
    M  [label="MEDICOS",       fillcolor="#2E7D52", fontcolor=white];
    C  [label="CITAS",         fillcolor="#2E7D52", fontcolor=white];
    D  [label="DIAGNOSTICOS", fillcolor="#2E7D52", fontcolor=white];
    H -> P; H -> M; H -> C; H -> D;
    p1 [label="María García\nA+ | Hab. 302",  fillcolor="#D4EDDA"];
    p2 [label="Carlos Mendoza\nO- | Hab. 410", fillcolor="#D4EDDA"];
    m1 [label="Dr. Roberto Juárez\nMED-001 | CARDIOLOGIA",
fillcolor="#D6EAF8"];
    m2 [label="Dra. Ana Solís\nMED-002 | NEUROLOGIA",
fillcolor="#D6EAF8"];
    P -> p1; P -> p2; M -> m1; M -> m2;
    p1 -> m1 [label="2025-04-10  09:00", color="#E67E22", style=dashed];
    p2 -> m2 [label="2025-04-10  11:30", color="#E67E22", style=dashed];
    d1 [label="Hipertensión Arterial\nLosartán 50mg / DIARIA",
fillcolor="#FDEBD0"];
    D -> d1; d1 -> p1 [label="diagnóstico activo", color="#C0392B"];
}
```

3.7 Requerimientos Técnicos

- Lenguaje: C++ estándar C++17 o superior.
- Interfaz gráfica: Qt 5.15 o Qt 6.x (recomendado) o wxWidgets 3.x. Componentes mínimos: área de texto para el código fuente, QTableWidget/wxGrid para tokens y errores, botones de carga y análisis, y acceso a los reportes HTML.
- El código debe organizarse en clases: Token (con enum TokenType), LexicalAnalyzer, ErrorManager, ReportGenerator y la clase principal de la GUI. Cada clase en su propio .h y .cpp.

- El proyecto debe compilar sin errores con g++ -std=c++17 o con QMake/CMakeLists.txt. El README debe incluir los comandos exactos de compilación paso a paso.
- Prohibido usar std::regex para reconocer tokens. El AFD debe implementarse en el método nextToken() con una función de transición de estados explícita.

3.8 Entregables

Tipo	Descripción
Código Fuente	Código completo en C++ organizado en clases y módulos, debidamente comentado. Debe incluir: clase Token (lexema, tipo, línea, columna), clase LexicalAnalyzer con método nextToken() basado en el AFD, clase ErrorManager, clase ReportGenerator y la clase principal de la GUI. El proyecto debe compilar con g++ -std=c++17 o con QMake/CMake.
Manual Técnico	Documento PDF con: arquitectura del sistema (diagrama de componentes UML), diagrama de clases completo, diagrama del AFD con todos los estados y transiciones etiquetadas, tabla de transiciones del autómata, descripción de algoritmos de tokenización y recuperación de errores, y justificación de decisiones de diseño.
Manual de Usuario	Guía con capturas de pantalla de la GUI que explique: cómo compilar e instalar dependencias, estructura del archivo .med con 2 ejemplos completos, cómo cargar y analizar un archivo, cómo interpretar la tabla de tokens y los mensajes de error, y cómo abrir y navegar cada uno de los 4 reportes HTML generados.
Repositorio GitHub	Repositorio privado con estructura /src /docs /tests /examples. Commits frecuentes y descriptivos. README.md con instrucciones de compilación paso a paso, dependencias y uso. .gitignore para C++/Qt. Auxiliar agregado como colaborador antes de la entrega.
Casos de Prueba	Documento con al menos 10 casos ejecutados: entradas válidas con las 4 secciones, errores de token inválido, cadenas sin cerrar, fechas con mes fuera de rango, horas inválidas, especialidades no reconocidas, múltiples errores simultáneos y casos borde. Cada caso con entrada, resultado esperado y resultado obtenido con captura.

4. Material de Apoyo

- Compilers: Principles, Techniques, and Tools (Dragon Book) — Aho, Sethi, Ullman, Lam. 2da edición. Capítulos 1, 2 y 3.
- Introduction to the Theory of Computation — Michael Sipser. 3ra edición. Capítulos 1 y 2.
- The C++ Programming Language — Bjarne Stroustrup. 4ta edición.
- Documentación oficial de Qt 6: https://doc.qt.io/qt-6/
- Documentación oficial de wxWidgets: https://docs.wxwidgets.org/stable/
- Graphviz DOT Language Reference: https://graphviz.org/doc/info/lang.html
- cppreference.com — Referencia completa de la STL de C++: https://en.cppreference.com

5. Metodología

El proyecto seguirá una metodología de desarrollo incremental con cuatro fases. Se recomienda completar y verificar cada fase antes de avanzar a la siguiente, ya que cada una depende de la anterior.

Fase 1 — Análisis del Lenguaje y Diseño del AFD (Semana 1)

- Definir formalmente la gramática léxica de MedLang: alfabeto, tipos de tokens, patrones y errores.
- Diseñar el diagrama completo del AFD con todos sus estados, transiciones y estados de aceptación.
- Elaborar la tabla de transiciones como referencia directa para la implementación en C++.
- Diseñar la arquitectura del sistema: clases, responsabilidades y relaciones.
- Definir los 10 casos de prueba preliminares.

Fase 2 — Implementación del Motor Léxico (Semana 2)

- Implementar la clase Token con el enum TokenType y sus atributos.
- Implementar la clase LexicalAnalyzer con nextToken() basado en el AFD diseñado.
- Implementar ErrorManager para acumular errores y permitir que el análisis continúe.
- Verificar el funcionamiento desde consola con todos los casos de prueba.

Fase 3 — Integración con GUI y Generación de Reportes (Semana 3)

- Diseñar e implementar la GUI en Qt o wxWidgets con todos los paneles requeridos.
- Integrar el motor léxico con la GUI para carga de archivos y visualización de resultados.
- Implementar ReportGenerator para los 4 reportes HTML y el código DOT de Graphviz.

Fase 4 — Pruebas, Corrección y Documentación (Semana 4)

- Ejecutar los 10 casos de prueba y documentar los resultados con capturas de pantalla.
- Corregir errores y verificar la cobertura del AFD con casos borde.
- Redactar el Manual Técnico con todos los diagramas requeridos.
- Redactar el Manual de Usuario con capturas de los 4 reportes.
- Organizar el repositorio GitHub y verificar que el proyecto compila desde cero siguiendo el README.

6. Recursos y Herramientas a Utilizar

Entorno de desarrollo:

- IDE: Qt Creator (recomendado), CLion, Visual Studio 2022 o VS Code con extensión C/C++.
- Compilador: g++ con soporte C++17 (MinGW en Windows, GCC en Linux/macOS) o MSVC 2019/2022.
- Sistema de construcción: QMake (.pro) o CMake 3.16+.

Frameworks y bibliotecas:

- Qt 5.15 o Qt 6.x para la GUI (QMainWindow, QTableWidget, QTextEdit, QFileDialog, QPushButton).
- Alternativamente: wxWidgets 3.x (wxFrame, wxGrid, wxTextCtrl, wxFileDialog).
- STL de C++17: std::vector, std::map, std::string, std::ifstream, std::fstream. Sin dependencias adicionales.

Herramientas de diseño:

- Draw.io o Lucidchart — para el diagrama del AFD, diagrama de clases UML y tabla de transiciones.
- Graphviz — para compilar y visualizar el archivo .dot generado por la aplicación.

Restricciones técnicas:

- Lenguaje: C++ (C++17 o superior). Interfaz: Qt 5/6 o wxWidgets 3.x.
- Tokenización únicamente con funciones nativas de std::string (at(), substr(), etc.). Prohibido std::regex.
- Prohibido el uso de librerías de análisis léxico o sintáctico (ANTLR, JFlex, Flex/Bison, etc.).

7. Desarrollo de Habilidades Blandas

Autogestión del tiempo

Se recomienda 6-8 horas semanales. La Fase 2 (AFD en C++) y la Fase 3 (integración con Qt) son las más demandantes. En C++ los ciclos de compilación y depuración toman más tiempo que en lenguajes interpretados; esto debe contemplarse en la planificación personal.

Responsabilidad y compromiso

El código, los diagramas y la documentación deben ser resultado del trabajo personal e individual. La prohibición de std::regex y de herramientas automáticas obliga al estudiante a comprender a fondo cada estado del AFD y cada transición implementada.

Resolución de problemas

El estudiante deberá resolver ambigüedades del lenguaje (distinguir entre un entero y el inicio de una fecha desde el mismo estado del AFD), implementar recuperación de errores robusta, y manejar cadenas con caracteres especiales como tildes y ñ.

Pensamiento crítico

El estudiante debe evaluar múltiples estrategias para representar los estados del AFD en C++ y justificar sus decisiones en el manual técnico con criterios de eficiencia, mantenibilidad y legibilidad del código.

8. Cronograma

El cronograma describe las etapas clave del proyecto y el proceso de asignación, elaboración y calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	03/03/2026	03/04/2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El proyecto debe desarrollarse en C++ (C++17 o superior) con interfaz gráfica funcional en Qt 5/6 o wxWidgets. No se permite el uso de std::regex para la tokenización principal ni de lenguajes alternativos.	
Uso de herramientas requeridas	Repositorio en GitHub con commits organizados, README con instrucciones de compilación, estructura de carpetas definida y .gitignore apropiado para C++/Qt.	
Gestión y entregas del proyecto	Entrega puntual del link al repositorio en UEDI con todos los entregables dentro: código compilable, manuales en PDF y casos de prueba documentados.	
Documentación obligatoria	Manual Técnico con diagrama de clases UML, diagrama del AFD y tabla de transiciones. Manual de Usuario con capturas de la GUI mostrando los 4 reportes. Ambos en PDF.	
Pruebas y funcionalidad mínima	El sistema debe: cargar archivos .med, tokenizar el lenguaje MedLang, generar tabla de tokens con posición exacta, reportar errores léxicos, y generar los 4 reportes HTML y el diagrama Graphviz.	

9.2 Resumen de Puntuaciones

Categoría	Punteo
Habilidades (Doc. Técnica, Manual de Usuario, Repositorio, Presentación, Casos de Prueba)	**40 pts**
Conocimientos (AFD, Análisis Léxico, Tablas, GUI, 4 Reportes HTML + Graphviz, Opcionales)	**60 pts**
TOTAL	**100 pts**

9.3 Rúbrica Detallada

No.	Criterio	Punteo	Rango	Satisfactorio (100%-61%)	Necesita Mejorar (60%-0%)
1	**Habilidades**	**40**			
1.1	**Documentación Técnica**	10	10-6	Manual técnico completo: arquitectura UML, diagrama de clases detallado, diagrama del AFD con todos los estados y transiciones, tabla de transiciones, descripción exhaustiva de algoritmos y justificación de decisiones de diseño.	Manual incompleto: faltan diagramas UML, el AFD no está documentado o es incorrecto, algoritmos sin describir, tabla de transiciones ausente.
1.2	**Documentación de Usuario**	8	8-5	Manual completo con instrucciones de	Instrucciones confusas, faltan capturas

				compilación e instalación de Qt/wxWidgets, capturas de la GUI, ejemplos de archivos .med válidos e inválidos, e interpretación de los 4 reportes HTML generados.	relevantes o no permite operar la aplicación de forma autónoma.
1.3	**Repositorio GitHub**	7	7-4	Repositorio organizado: /src /docs /tests /examples, commits frecuentes y descriptivos, README con instrucciones de compilación, .gitignore para C++/Qt y auxiliar como colaborador.	Repositorio desorganizado, pocos commits o genéricos, README ausente o sin instrucciones de compilación.
1.4	**Presentación**	8	8-5	Presentación profesional: contexto del problema, arquitectura del sistema, demostración en vivo con la GUI, explicación del AFD, ejecución de casos de prueba en tiempo real y conclusiones técnicas.	Sin demostración funcional o el estudiante no puede explicar el AFD ni las decisiones de diseño.
1.5	**Casos de Prueba**	7	7-4	Al menos 10 casos ejecutados y documentados: válidos, errores de tipo inválido, cadenas sin cerrar, fechas incorrectas, especialidades no reconocidas, múltiples errores y casos borde. Cada caso con entrada, resultado esperado y obtenido.	Menos de 6 casos, mal documentados o sin evidencia de ejecución.
2	**Conocimientos**	**60**			
2.1	**Diseño e Implementación del AFD**	15	15-9	AFD implementado manualmente en C++ con función de transición de estados explícita (sin std::regex). Reconoce todos los tokens MedLang: palabras reservadas, códigos ID, fechas AAAA-MM-DD, horas HH:MM, enteros, cadenas, enumeraciones y delimitadores. Tracking preciso de línea y columna.	Usa std::regex para tokenización, estados faltantes o incorrectos, tracking de posición impreciso o ausente.

2.2	**Análisis Léxico — Reconocimiento**	10	10-6	Reconoce el 95-100% de los tokens en los casos de prueba. Procesa carácter por carácter. Maneja delimitadores, espacios y saltos de línea correctamente. Continúa el análisis tras errores léxicos sin detenerse.	Reconoce menos del 80%, se detiene ante el primer error léxico o no maneja todos los delimitadores.
2.3	**Tabla de Tokens y Errores Léxicos**	10	10-6	Tabla de tokens en GUI y HTML con: número, lexema, tipo, línea y columna (precisión 100%). Tabla de errores con: número, lexema, tipo de error, descripción y posición exacta. Ambas visualmente claras.	Tablas incompletas, posición incorrecta frecuentemente o no se generan ambas tablas.
2.4	**Interfaz Gráfica (GUI)**	10	10-6	GUI funcional en Qt o wxWidgets: área de código fuente, panel de tokens, panel de errores, botones de carga y análisis, y acceso a los 4 reportes HTML. Navegación fluida y diseño coherente.	GUI incompleta, botones no funcionales o diseño confuso que impide visualizar los resultados.
2.5	**Los 4 Reportes HTML y Graphviz**	12	12-7	Genera los 4 reportes HTML requeridos con estadísticas correctas y con el formato descrito en la sección 3.4: Pacientes, Médicos, Agenda de Citas y Estadístico General. El código DOT produce un diagrama válido con la jerarquía del hospital.	Genera menos de 3 reportes, estadísticas incorrectas, formato incompleto o el código DOT no produce salida válida.
2.6	**Funcionalidades Opcionales**	3	3-2	Implementa al menos una funcionalidad opcional correctamente: resaltado de sintaxis en la GUI, exportación a CSV/JSON, detección de conflictos de horario, estadísticas del análisis (tiempo en ms, frecuencia de tokens) o soporte multi-archivo.	No implementa opcionales o las implementadas están incompletas y no aportan valor.

10. Entrega

- Subir el link al repositorio de GitHub en el apartado de UEDI antes de la fecha y hora límite.
- El repositorio debe contener:
  - /src — Código fuente en C++ y archivos .pro o CMakeLists.txt
  - /docs — Manual Técnico (PDF) y Manual de Usuario (PDF)
  - /tests — Archivos .med de casos de prueba (mínimo 10, válidos e inválidos)
  - /examples — Al menos 2 archivos .med completos de ejemplo
  - README.md — Instrucciones de compilación, dependencias y uso
  - .gitignore — Para proyectos C++/Qt
- Nombre del repositorio: LFP_1S2026_#Carnet. Carpeta interna: Proyecto1.
- Agregar al auxiliar de la sección como colaborador antes de la entrega.
- La calificación es PRESENCIAL. El estudiante debe demostrar la GUI en funcionamiento, cargar un archivo .med en tiempo real y explicar el diseño del AFD.

Fecha límite: 3 de abril, antes de las 23:59 hrs. No se reciben entregas fuera del plazo.

EN CASO DE ENCONTRAR COPIAS PARCIALES O TOTALES DEL CÓDIGO O DOCUMENTACIÓN, SE ANULARÁ EL PROYECTO AUTOMÁTICAMENTE, SE NOTIFICARÁ AL CATEDRÁTICO Y SE REPORTARÁ A LA ESCUELA DE CIENCIAS Y SISTEMAS PARA LAS SANCIONES CORRESPONDIENTES.